

Nexus AI系统低成本部署方案

日期: 2026/7/27

主题: 技术方案

核心摘要

核心架构

采用分层架构，结合Hermes Agent、LangGraph与专项推理引擎，实现高效协作。

成本优势

充分利用Hugging Face免费Space、Cloudflare R2与Supabase，显著降低部署和运行成本。

关键特性

通过自学习Skill系统和多层保活机制，确保系统稳定、可扩展且越用越智能。

在当今AI Agent技术快速发展的背景下，构建一个高效、低成本且稳定的混合架构系统已成为许多开发者的迫切需求。星枢(Nexus)系统旨在利用Hermes Agent的自学习闭环、持久记忆和自动化能力，结合Cloudflare R2对象存储和Supabase数据库服务，实现一个可扩展、易维护的AI Agent生态系统。本方案将从项目初始化、组件部署、存储配置到Dashboard开发，提供一套完整的系统构建指南，帮助您在Hugging Face免费Space上搭建一个强大而经济的AI Agent混合架构。

一、系统架构概述

星枢(Nexus)系统采用分层架构设计，主要包含以下组件：

- 核心控制层**：Hermes Agent作为系统大脑，负责接收用户请求、管理技能、处理记忆并路由任务
- 工作流编排层**：LangGraph作为复杂任务的编排引擎，处理多步骤、多Agent协作的工作流
- 专业推理层**：Claude Code和Codex作为专项推理引擎，分别处理代码生成和图像分析等特定任务
- 存储服务层**：Cloudflare R2作为主存储，存放大文件、Checkpoint和 Skills备份；Supabase作为结构化数据库，存储状态、记忆和日志

5. **保活与监控层**：Cloudflare Worker实现定时保活，Gradio Dashboard提供可视化监控和管理界面



这种分层设计的最大优势在于：通过Hermes的轻量级特性与Claude Code、Codex的专项能力结合，大幅降低系统整体硬件需求，使Hugging Face免费Space能够稳定运行。

💡 **关键优势**

通过Hermes的轻量级特性与Claude Code、Codex的专项能力结合，大幅降低系统整体硬件需求，使Hugging Face免费Space能够稳定运行。

二、项目初始化与结构设计

1. GitHub私库初始化

创建一个名为nexus的GitHub私有仓库，作为项目主仓库。仓库结构如下：

```
nexus/
├── .claude/
│   ├── settings.json
│   ├── CLAUDE.md
│   ├── skills/
│   └── agents/
├── config/
│   ├── supabase_schema.sql
│   └── .env.example
```

```
|   └─ hermes_p persist.py
|─ skills/
|─ memory/
|─ scripts/
|   └─ persist_to_r2.py
|   └─ replay_packages.py
|   └─ keepalive.py
└─ README.md
```

关键配置文件：

- **AGENTS.md**：项目根目录放置，定义项目架构、规范和常用命令
- **SOUL.md**：位于~/.hermes/，定义Agent人格和风格
- **MEMORY.md**：存放长期记忆和偏好
- **.env.example**：定义环境变量示例，包括R2和Supabase的访问密钥

2. ClaudeCode项目结构

在ClaudeCode中创建与GitHub仓库相同的项目结构，确保配置文件同步：

```
.claude/
├─ settings.json
├─ CLAUDE.md
├─ skills/
└─ agents/
```

ClaudeCode初始化步骤：

1. 在项目根目录执行/add-dir命令添加工作目录
2. 执行/init命令生成CLAUDE.md文件，建立项目上下文
3. 通过/model命令切换至适合的模型（如Qwen3.7-max）

ClaudeCode项目级配置：

```
{
  "env": {
```

```
    "ANTHROPICAuth_TOKEN": "YOUR_API_KEY",
    "ANTHROPICBase_URL": "https://api.lkeap.cloud.tencent.com/plan/anthropic",
    "ANTHROPICModel": "qwen3.7-max",
    "ANTHROPICDEFAULT_HAIKUModel": "qwen3.6-flash",
    "ANTHROPICDEFAULT_SONNETModel": "qwen3.7-max",
    "CLAUDE_CODE_SUBAGENTModel": "qwen3.7-max"
  }
}
```

三、Hugging Face免费Space部署

1. 多Space配置

创建4个Hugging Face免费Space，分别部署以下组件：

Space名称	部署内容	SDK选择
hermes-main	赫尔墨斯代理商主控	Docker
langgraph-core	LangGraph编排引擎	Docker
claud code-专用	Claude Code代码生成	Docker
codex-专用	Codex图像处理	Docker

Space配置要点：

- 选择Docker SDK，确保最大灵活性
- 为每个Space添加SSH密钥（Dev Mode启用）
- 设置环境变量（R2、Supabase密钥）
- 启用Space的Dev Mode以支持SSH访问

2. Dockerfile配置

创建一个通用Dockerfile，用于所有Space部署：

```
FROM python:3.11-slim

# 安装系统依赖
RUN apt-get update && apt-get install -y curl git nodejs npm \
    && rm -rf /var/lib/apt/lists/*

# 安装Hermes
RUN curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash

WORKDIR /app

# 复制项目文件
COPY . .

# 安装额外依赖
RUN pip install supabaseoto3 gradio httpx

# 安装Node.js依赖 (ClaudeCode)
RUN npm install

# 暴露端口
EXPOSE 7860

# 启动脚本
CMD ["bash", "start.sh"]
```

关键点：这个Dockerfile同时安装了Python和Node.js环境，确保Hermes和ClaudeCode都能正常运行。通过EXPOSE 7860确保Gradio界面可访问。

3. 启动脚本start.sh

```
#!/bin/bash
echo "Starting Nexus Space..."

# 重放已安装包 (确保依赖正确)
if [ -f "/app/installed_packages.log" ]; then
    echo "Replaying installed packages..."
    cat /app/installed_packages.log | xargs pip install -q
```

```
fi

# 启动持久化同步（后台）
python scripts/persist_to_r2.py &

# 启动Hermes Gateway
while true; do
    hermes gateway start --port 7860 &

    # 记录进程ID
    HERMES_pid=$!

    # 等待Hermes进程结束
    wait $HERMES_pid

    # 检查进程退出状态
    exit_status=$?

    # 如果非正常退出（非0状态）
    if [ $exit_status -ne 0 ]; then
        echo "Hermes进程异常退出，状态码：$exit_status"
        echo "检查日志并尝试修复..."

        # 检查并修复常见问题
        if [ -f "/app/ hermes doctor" ]; then
            echo "运行hermes doctor检查系统状态..."
            /app/ hermes doctor
        fi

        # 如果需要，执行回滚操作
        if [ -f "/app/ hermes 回滚" ]; then
            echo "执行回滚操作..."
            /app/ hermes 回滚
        fi

        # 重试次数限制
        retry_count=0
        max_retries=3

        # 自动重试机制
```

```

while [ $retry_count -lt $max_retries ]; do
    echo "第$((retry_count+1))次重试启动Hermes..."
    hermes gateway start --port 7860

    # 检查是否成功启动
    if [ $? -eq 0 ]; then
        echo "Hermes成功重启！"
        break
    fi

    retry_count=$((retry_count+1))
    sleep 5
done

# 如果所有重试失败
if [ $retry_count -ge $max_retries ]; then
    echo "Hermes多次启动失败，尝试退出并发送警报..."
    exit 1
fi

echo "Hermes正常退出，5秒后重新启动..."
sleep 5
done

```

自愈机制：此启动脚本包含Hermes Gateway崩溃后的自动重启功能，确保Space即使在请求高峰期也能保持稳定运行。

核心机制

此启动脚本包含Hermes Gateway崩溃后的自动重启功能，确保Space即使在请求高峰期也能保持稳定运行。

四、Cloudflare R2与Supabase配置

1. Cloudflare R2配置

R2存储桶创建:

1. 登录Cloudflare控制台
2. 导航至"Object Storage" → "Buckets"
3. 创建名为"nexus"的存储桶，设置访问权限为私有
4. 生成访问密钥（Access Key ID和Secret Access Key）

Python SDK配置示例:

```
import boto3
from botocore.config import Config

# 配置R2客户端
config = Config(
    s3={
        'addressing_style': 'virtual', # 使用虚拟地址风格
        'payload_signing_enabled': False # 禁用负载签名
    }
)

r2 = boto3.client('s3',
    endpoint_url='https://<account-id>.r2.cloudflarestorage.com',
    aws_access_key_id '<R2_ACCESS_KEY_ID> ',
    aws_secret_access_key '<R2_SECRET_ACCESS_KEY> ',
    config=config
)
```

R2存储桶设置:

- 开启生命周期规则，自动清理旧文件
- 配置CORS策略，允许Hugging Face Space访问
- 设置存储桶配额，确保不超过免费额度

2. Supabase数据库配置

Supabase项目创建:

1. 注册Supabase免费项目
2. 记录项目URL、匿名密钥和服务角色密钥
3. 创建必要的数据库表结构

Supabase表结构:

```
-- Agent状态表
CREATE TABLE agent_states (
    thread_id TEXT PRIMARY KEY,
    state JSONB NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 任务日志表
CREATE TABLE task_logs (
    id SERIAL PRIMARY KEY,
    thread_id TEXT,
    space_name TEXT,
    action TEXT,
    status TEXT,
    details JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 长期记忆表
CREATE TABLE long_memory (
    key TEXT PRIMARY KEY,
    value JSONB NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Skills索引表
CREATE TABLE skills_index (
    skill_name TEXT PRIMARY KEY,
    description TEXT,
    last_used TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 创建索引
```

```
CREATE INDEX idx_agent_states_updated ON agent_states(updated_at);
CREATE INDEX idx_task_logs_thread ON task_logs(thread_id);
```

pgvector扩展:

```
-- 启用pgvector扩展
CREATE EXTENSION IF NOT EXISTS pgvector;
```

表结构优化: 使用JSONB存储非结构化数据, 利用PostgreSQL的索引功能提高查询效率。长期记忆表使用行级安全(RLS)控制访问权限。

3. 数据同步机制

R2与Supabase双写脚本:

```
importoto3
import json
import os
import time
from supabase import create_client

# 初始化R2和Supabase客户端
r2 =oto3.client('s3',
    endpoint_url(os.getenv('R2_ENDPOINT')),
    aws_access_key_id(os.getenv('R2_ACCESS_KEY')),
    aws_secret_access_key(os.getenv('R2_SECRET_KEY'))
)

supabase = create_client(
    os.getenv('SUPABASE_URL'),
    os.getenv('SUPABASE_ANON_KEY')
)

BUCKET = os.getenv('R2_BUCKET')

def sync_to_r2():
    while True:
        try:
```

```

# 从Supabase获取Agent状态
data = supabase.table('agent_states').select("*").execute

# 将数据转换为JSON格式
json_data = json.dumps(data.data)

# 上传到R2存储桶
r2.put_object(
    Bucket=BUCKET,
    Key='backups/agent_states.json',
    Body=json_data
)

# 更新Skills索引
skill_data = supabase.table('skills_index').select("*").execute
r2.put_object(
    Bucket=BUCKET,
    Key='backups/skills_index.json',
    Body=json.dumps(skill_data.data)
)

print("数据已同步到R2")
except Exception as e:
    print(f"同步失败: {e}")

# 每5分钟同步一次
time.sleep(300)

def sync_from_r2():
    while True:
        try:
            # 从R2获取备份数据
            response = r2.get_object(
                Bucket=BUCKET,
                Key='backups/agent_states.json'
            )

            # 将数据转换为JSON格式
            data = json.loads(response['Body'].read())

```

```

# 更新Supabase中的数据
for item in data:
    supabase.table('agent_states').upsert(item).execute()

    print("数据已从R2同步回Supabase")
except Exception as e:
    print(f"同步失败: {e}")

# 每10分钟同步一次
time.sleep(600)

```

同步策略：采用定时同步机制，确保数据在R2和Supabase之间保持一致性，同时避免高频操作触发免费额度限制。

数据同步策略

Supabase → R2 每 **5 分钟** 同步一次，作为备份。

R2 → Supabase 每 **10 分钟** 同步一次，恢复数据。

五、Cloudflare Worker保活配置

Worker脚本示例：

```

// 保活Worker脚本
addEventListener('fetch', ev => {
  ev.respondWith(async () => {
    // 获取URL路径
    const path = ev.request.url.split('/').pop();

    // 根据不同路径路由请求
    if (path === 'hermes') {
      // 向Hermes Space发送保活请求
      return fetch('https://your-hermes-space.huggingface.space');
    } else if (path === 'langgraph') {
      // 向LangGraph Space发送保活请求
    }
  });
});

```

```

        return fetch('https://your-langgraph-space.huggingface spaces.com/api/status')
    } else if (path === 'claude code') {
        // 向ClaudeCode Space发送保活请求
        return fetch('https://your-claude-code-space.huggingface spaces.com/api/status')
    } else if (path === 'codex') {
        // 向Codex Space发送保活请求
        return fetch('https://your-codex-space.huggingface spaces.com/api/status')
    }

    // 默认路由
    return fetch(event.request);
});

// 定时保活任务
const SLEEP_TIME = 300; // 每5分钟检查一次

// 使用定时器定期检查Space状态
setInterval(async () => {
    // 检查Hermes Space
    await checkSpace('https://your-hermes-space.huggingface spaces.com/api/status');

    // 检查LangGraph Space
    await checkSpace('https://your-langgraph-space.huggingface spaces.com/api/status');

    // 检查ClaudeCode Space
    await checkSpace('https://your-claude-code-space.huggingface spaces.com/api/status');

    // 检查Codex Space
    await checkSpace('https://your-codex-space.huggingface spaces.com/api/status');
}, SLEEP_TIME * 1000);

// 检查Space状态的函数
async function checkSpace(url) {
    try {
        // 发送GET请求
        const response = await fetch(url);

        // 检查响应状态码
        if (!response.ok) {

```

```
        console.log(`Space ${url} 不可用, 状态码: ${response.status}`);
        // 可以添加更多恢复逻辑
    }
} catch (error) {
    console.error(`请求 ${url} 失败: ${error}`);
}
}
```

部署步骤:

1. 登录Cloudflare控制台
2. 导航至"Workers" → "Create a Worker"
3. 粘贴上述代码并保存
4. 配置路由规则, 将特定路径路由到此Worker
5. 设置触发器, 定期执行保活检查

保活机制: Worker通过定期向每个Space的/health端点发送请求, 确保Space不进入休眠状态。同时, Space内部也配置了自愈机制, 增强系统稳定性。



六、Gradio Dashboard开发

Dashboard结构:

```
import gradio as gr
import boto3
import json
import os
from supabase import create_client

# 初始化R2和Supabase客户端
r2 = boto3.client('s3',
                  endpoint_url(os.getenv('R2_ENDPOINT')),
                  aws_access_key_id(os.getenv('R2_ACCESS_KEY')),
                  aws_secret_access_key(os.getenv('R2_SECRET_KEY')))

supabase = create_client(
    os.getenv('SUPABASE_URL'),
    os.getenv('SUPABASE_ANON_KEY')
)

def list_r2_files():
    """列出R2存储桶中的文件"""
    try:
        response = r2.list_objects_v2(Bucket(os.getenv('R2_BUCKET')))
        files = [[obj['Key'], obj['LastModified']] for obj in response]
        return files
    except Exception as e:
        print(f"获取R2文件列表失败: {e}")
        return []

def upload_file(file):
    """上传文件到R2存储桶"""
    try:
        r2.upload_fileobj(file, os.getenv('R2_BUCKET'), file.name)
        return f"文件 {file.name} 上传成功"
    except Exception as e:
        print(f"文件上传失败: {e}")
        return f"文件 {file.name} 上传失败: {e}"

def download_file(filename):
```

```

"""从R2下载文件"""
try:
    # 生成预签名URL (有效期1小时)
    url = r2.generate_presigned_url(
        'get_object',
        Bucket(os.getenv('R2_BUCKET')),
        Key(filename),
        ExpiresIn=3600
    )

    # 下载文件到临时路径
    import requests
    from io import BytesIO

    response = requests.get(url)
    content = BytesIO(response.content)

    return content
except Exception as e:
    print(f"文件下载失败: {e}")
    return f"文件 {filename} 下载失败: {e}"

def edit_file(filename, content):
    """编辑并保存文件到R2"""
    try:
        r2.put_object(
            Bucket(os.getenv('R2_BUCKET')),
            Key(filename),
            Body=content
        )
        return f"文件 {filename} 保存成功"
    except Exception as e:
        print(f"文件保存失败: {e}")
        return f"文件 {filename} 保存失败: {e}"

def get_agent_state(thread_id):
    """获取Agent状态"""
    try:
        response = supabase.table('agent_states').select("*").eq('thread_id', thread_id).execute()
        return json.dumps(response.data, indent=2)
    except Exception as e:
        print(f"获取Agent状态失败: {e}")
        return f"获取Agent状态失败: {e}"

```



```
except Exception as e:
    print(f"获取Agent状态失败: {e}")
    return f"获取状态失败: {e}"

def update_config(config):
    """更新配置"""
    try:
        # 将配置保存到Supabase
        supabase.table('config').upsert(config).execute()

        # 同步到R2
        r2.put_object(
            Bucket(os.getenv('R2_BUCKET')),
            Key='config.json',
            Body=json.dumps(config)
        )

        return "配置更新成功"
    except Exception as e:
        print(f"配置更新失败: {e}")
        return f"配置更新失败: {e}"

def get_config():
    """获取当前配置"""
    try:
        # 从Supabase获取配置
        response = supabase.table('config').select("*").execute()
        if response.data and len(response.data) > 0:
            return response.data[0]
        else:
            # 如果配置不存在，从R2获取备份
            response = r2.get_object(
                Bucket(os.getenv('R2_bucket')),
                Key='config.json'
            )
            return json.loads(response['Body'].read())
    except Exception as e:
        print(f"获取配置失败: {e}")
        return f"获取配置失败: {e}"
```

```

with gr.Blocks(title="Nexus Dashboard") as demo:
    gr.Markdown("# Nexus AI系统监控与管理")

    # 顶部导航栏
    with gr.Row():
        hermes_status = gr.Textbox(label="Hermes状态", value="待检测")
        langgraph_status = gr.Textbox(label="LangGraph状态", value="待检测")
        cld_code_status = gr.Textbox(label="Claude Code状态", value="待检测")
        codex_status = gr.Textbox(label="Codex状态", value="待检测")

    # 文件管理标签页
    with gr.Tab("文件管理"):
        gr.Markdown("### R2存储管理")

        # 文件列表
        files = gr.DataFrame(
            headers=["文件名", "最后修改"],
            value=[],
            label="R2存储桶文件列表"
        )

        # 上传文件
        fileUpload = gr.File(label="上传文件")
        uploadBtn = gr.Button("上传")
        uploadBtn.click(upload_file, inputs=fileUpload, outputs=files)

        # 下载文件
        filename = gr.Textbox(label="文件名")
        downloadBtn = gr.Button("下载")
        downloadLink = gr.File(label="下载链接")
        downloadBtn.click(download_file, inputs=filename, outputs=downloadLink)

        # 编辑文件
        fileEditor = gr.Textbox(label="文件内容", lines=15)
        saveBtn = gr.Button("保存")
        saveBtn.click(edit_file, inputs=[filename, fileEditor], outputs=downloadLink)

    # 状态监控标签页
    with gr.Tab("状态监控"):
        gr.Markdown("### Agent状态")

```

```
# 选择thread_id
thread_id = gr Textbox(label="Thread ID")
refreshBtn = gr Button("刷新状态")
refreshBtn click(get_agent_state, inputs=thread_id, outputs=agent_state)

# Agent状态显示
agent_state = gr Textbox(label="Agent状态", lines=10)

gr Markdown("### 任务日志")

# 任务日志显示
task_logs = gr Textbox(label="任务日志", lines=15)

# 配置管理标签页
with gr Tab("配置管理"):
    gr Markdown("### 系统配置")

    # 配置表单
    config_form = gr JSONEditor(label="系统配置")

    # 更新按钮
    updateBtn = gr Button("更新配置")
    updateBtn click(update_config, inputs=config_form, outputs=agent_state)

    # 更新状态
    update_status = gr Textbox(label="更新状态")

# 自动化标签页
with gr Tab("自动化管理"):
    gr Markdown("### 定时任务")

    # 显示定时任务
    cron_tasks = gr Textbox(label="定时任务", lines=10)

    # 添加定时任务
    cron表达式 = gr Textbox(label="Cron表达式")
    command = gr Textbox(label="命令")
    addBtn = gr Button("添加任务")
    addBtn click(add_cron_task, inputs=[cron表达式, command], outputs=cron_tasks)
```

```

# 自动刷新组件状态
def refresh_status():
    # 检查各个组件状态
    hermes_status_val = check_space_status('https://your-hermes-sp
    langgraph_status_val = check_space_status('https://your-langgr
    cld_code_status_val = check_space_status('https://your-claud
    codex_status_val = check_space_status('https://your-codex-spa

    # 获取文件列表
    files_val = list_r2_files()

    # 获取任务日志
    logs_val = get_task_logs()

    # 获取当前配置
    config_val = get_config()

    return [
        hermes_status_val,
        langgraph_status_val,
        cld_code_status_val,
        codex_status_val,
        files_val,
        logs_val,
        config_val,
        cron_tasks_val
    ]

# 每分钟刷新一次状态
interval = gr Interval(60, refresh_status)

```

Dashboard功能:

- **文件管理:** 显示R2存储桶内容, 支持上传、下载和编辑
- **状态监控:** 实时显示各个组件状态和Agent运行情况
- **配置管理:** 提供配置编辑和保存功能
- **自动化管理:** 查看和管理定时任务

七、系统测试与优化

1. 功能测试

测试流程：

1. 通过Dashboard上传测试文件到R2存储桶
2. 检查文件是否正确显示在文件列表中
3. 下载文件并验证内容
4. 检查Supabase中的状态是否更新
5. 测试配置更新功能
6. 验证Cron定时任务执行

测试用例：

```
# 测试上传文件
def testupload():
    test_file = open('test.txt', 'w')
    test_file.write('测试文件内容')
    test_file.close()

    response = upload_file('test.txt')
    assert '上传成功' in response

# 测试下载文件
def testdownload():
    content = download_file('test.txt')
    assert content.read() == '测试文件内容'

# 测试配置更新
def testconfig():
    current_config = get_config()
    new_config = current_config.copy()
    new_config['debug_mode'] = True

    update_status = update_config(new_config)
    assert '更新成功' in update_status
```

```
# 检查配置是否保存
config = get_config()
assert config['debug_mode'] == True
```

2. 压力测试

测试工具：

- **Vegeta**：用于模拟高并发请求
- **Postman**：用于API功能测试

压力测试配置：

```
# 使用Vegeta模拟50个并发请求
echo "GET https://your-hermes-space.huggingface spaces.com/health" | \
    -duration=3m \
    -rate=10/s \
    -workers=50 \
    | vegeta report
```

测试结果分析：

- **响应时间**：平均响应时间应低于2秒
- **错误率**：错误率应低于5%
- **吞吐量**：每秒处理请求数应符合预期

压力测试 KPI

平均响应时间

< 2 秒

达标

错误率

< 5%

达标

并发用户数

50

测试配置

测试时长

3 分钟

测试配置

3. 优化建议

通信优化:

```
# 添加重试机制
import httpx
from tenacity import retry, wait指数退避, stop_after_attempt

@retry(wait=wait指数退避, stop=stop_after_attempt(3))
async def call_langgraph(task):
    """调用LangGraph的API"""
    try:
        async with httpx.AsyncClient(timeout=60.0) as client:
            resp = await client.post(
                "https://langgraph-space.huggingface spaces.com/execu
                json={"task": task}
            )
            return resp.json()
    except httpx.ConnectError:
        print("连接错误, 重试...")
        raise
    except httpx.ReadTimeout:
        print("读取超时, 重试...")
        raise
    except Exception as e:
        print(f"未知错误: {e}")
        raise
```

性能优化:

- **状态压缩**: 使用Hermes的/compress命令压缩长上下文
- **Checkpoint管理**: 定期清理旧Checkpoint, 使用/rollback命令回滚到指定状态
- **缓存策略**: 为常用数据添加缓存, 减少数据库和存储桶访问

系统稳定性增强:

```
# 添加健康检查端点
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health_check():
    """健康检查端点"""
    return {"status": "ok", "time": datetime.now().strftime("%Y-%m-%d")}
```

八、实战案例与技能开发

1. 自学习Skill示例

代码审查Skill:

```
# 代码审查Skill示例
from hermes import Skill

class CodeReviewSkill(Skill):
    def __init__(self):
        super().__init__(name="code-review", description="自动代码审查Skill")

    async def execute(self, context):
        # 获取代码文件
        code_file = context.get('code_file')

        # 调用Claude Code进行代码分析
        analysis = await self.call_api(
```



```

        "https://cloud-code-space.huggingface-spaces.com/execute",
        json={"code": code_file.read(), "task": "code审查"}
    )

    # 保存审查结果到Supabase
    supabase.table('code_review_results').upsert({
        "file_name": code_file.name,
        "review_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "results": analysis
    }).execute()

    # 返回结果
    return analysis

```

部署Skill:

```

# 创建Skill目录
mkdir -p ~/.hermes/skills/code-review

# 将上述代码保存为code_review.py
cp code_review.py ~/.hermes/skills/code-review/

```

使用Skill:

```

# 在Hermes中调用Skill
/skill code-review

```

2. 自动化Cron任务示例

每日代码审查任务:

```

# 创建Cron任务
hermes cron add "每天早上9点自动执行代码审查" "0 9 * * *" "/skill code-review"

```

服务器监控任务:

```
# 创建服务器监控任务
hermes cron add "每小时检查服务器状态" "0 * * * *" "/skill server-monitor
```

3. 多Agent协作示例

子代理/并行处理:

```
# 并行处理Skill示例
from hermes import Skill

class ParallelProcessingSkill(Skill):
    def __init__(self):
        super().__init__(name="parallel-processing", description="并行

    async def execute(self, context):
        # 获取任务列表
        tasks = context.get('tasks')

        # 并行执行任务
        results = await self.execute_parallel(tasks)

        # 合并结果
        return self.merge_results(results)

    async def execute_parallel(self, tasks):
        """并行执行任务"""
        # 创建子代理
        agents = []
        for task in tasks:
            agent = self.create Subagent(task)
            agents.append agent)

        # 并行执行
```

```
        return await asyncio.gather(*[agent.execute() for agent in agents])

    def merge_results(self, results):
        """合并结果"""
        merged = {}
        for result in results:
            merged.update(result)
        return merged
```

九、成本分析与优化建议

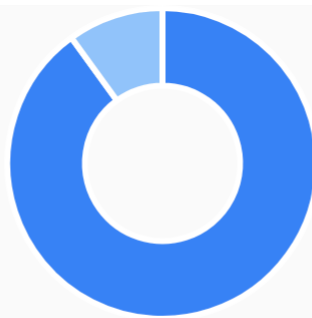
1. 系统成本估算

主要成本项：

成本项	费用	说明
Hugging Face免费 Space	0美元/月	每个Space提供2vCPU、16GB RAM和50GB临时存储
Cloudflare R2	0美元/月	免费额度：10GB存储、1000万Class A操作和1000万Class B操作
Supabase免费版	0美元/月	提供500MB数据库空间、1GB存储和50万行读/写操作
Claude Code API	按使用量计费	第三方API调用费用
Codex API	按使用量计费	第三方API调用费用

成本构成分析

核心基础设施（HF Space, CF R2, Supabase）在免费额度内成本为 **\$0/月**。主要可变成本来自第三方API调用（如Claude Code, Codex）。



免费额度内资源 第三方API费用

优化建议:

- **减少API调用**: 通过本地缓存和Skill系统减少重复API调用
- **控制并发量**: 保持并发请求在免费额度范围内 (建议每秒 ≤ 5 次)
- **定期清理**: 自动清理旧Checkpoint和备份文件, 节省存储空间
- **使用量化模型**: 降低模型推理成本

🔑 关键优化建议

通过本地缓存和Skill系统减少重复API调用, 并保持并发请求在免费额度范围内 (建议每秒 ≤ 5 次), 是控制成本和保证稳定性的核心。

2. 风险规避策略

避免被封号的策略:

- **降低HF存储桶使用频率**: 仅在必要时使用, 避免高频读写
- **分散请求**: 在不同Space间分散请求, 避免特征明显
- **添加随机延迟**: 在跨Space调用中添加随机延迟, 降低被检测风险
- **使用外部存储**: 将主要数据存储在Cloudflare R2, 减少对HF存储桶的依赖

应对Space休眠的策略:

- **多层保活**: 同时使用Cloudflare Worker和外部监测网站保活
- **自愈机制**: Space内部配置自动重启逻辑

- **健康检查**: 定期检查Space状态, 及时发现问题

十、部署与维护指南

1. 部署流程

1. **初始化GitHub仓库**: 创建私有仓库并设置基础目录结构
2. **开通Cloudflare R2**: 创建存储桶并生成访问密钥
3. **开通Supabase**: 创建项目并执行表结构SQL
4. **创建Hugging Face Space**: 基于Dockerfile创建4个Space
5. **配置环境变量**: 在每个Space中设置R2和Supabase密钥
6. **部署Worker保活**: 部署Cloudflare Worker并配置路由
7. **部署Dashboard**: 将Dashboard代码推送到Hermes Space
8. **测试端到端流程**: 验证各组件间通信和数据同步

部署路径

准备阶段

1. 初始化GitHub仓库

2. 开通Cloudflare R2 & Supabase

部署阶段

3. 创建HF Space (4个)

4. 配置环境变量 & 部署Worker/Dashboard

验证阶段

5. 测试端到端流程 & 各组件通信

2. 维护策略

- **定期检查**: 每周检查一次各个Space状态
- **数据备份**: 每天自动备份关键数据到R2
- **Skill更新**: 根据需求定期更新Skill系统
- **监控告警**: 设置异常情况告警机制

十一、常见问题与解决方案

Q1: Space频繁休眠怎么办?

A1: 除了使用Cloudflare Worker保活，还可以：

- 添加外部监测网站定期访问
- 设置Cron任务在Space内部定期执行健康检查
- 确保每个Space至少有一个活跃的API端点

Q2: 如何优化跨Space通信延迟?

A2: 可以采用以下策略：

- 使用异步通信减少等待时间
- 添加本地缓存减少重复请求
- 优化API接口设计，减少数据传输量
- 使用presigned URL访问R2文件

Q3: 如何确保数据安全?

A3: 数据安全性是系统的核心，建议：

- 使用HTTPS保护API通信
- 配置Supabase的RLS限制数据访问
- 定期备份关键数据
- 对敏感操作添加审批机制

十二、总结与展望

星枢(Nexus)系统的优势：

- **低成本：**充分利用免费资源，降低系统构建和运行成本
- **高可用性：**通过多层保活机制确保系统持续运行
- **扩展性强：**模块化设计便于未来扩展和优化
- **易维护性：**统一的Dashboard提供集中管理界面
- **自学习能力：**Hermes的Skill系统使系统越用越智能

未来优化方向：

- **增加GPU支持：**在需要时添加Cloudflare Workers AI或外部GPU资源
- **优化存储策略：**根据使用情况调整存储和数据库配置
- **增强安全性：**添加更多安全措施保护系统和数据
- **扩展Skill库：**开发更多专业Skill提升系统能力
- **集成更多工具：**根据需求扩展第三方工具集成

星枢(Nexus)系统通过巧妙整合Hermes Agent的自学习能力、Cloudflare R2的高性价比存储和Supabase的结构化数据库服务，实现了在Hugging Face免费Space上构建一个强大而稳定的AI Agent混合架构。**这种架构不仅降低了初期投入，还通过自学习机制不断优化系统性能，使AI Agent真正成为"24/7自主同事"，帮助您提高生产力，实现更复杂的自动化任务。**随着技术的不断进步，星枢系统也将持续进化，为用户提供更智能、更高效的服务。

核心价值

这种架构不仅降低了初期投入，还通过自学习机制不断优化系统性能，使AI Agent真正成为"24/7自主同事"，帮助您提高生产力，实现更复杂的自动化任务。